

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Модели данных и системы управления базами данных

К защите допустить:

И.О. Заведующего кафедрой
информатики

_____ С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту на тему

**МОБИЛЬНОЕ ПРИЛОЖЕНИЕ НА ПЛАТФОРМЕ ANDROID ДЛЯ
ЛИЧНОГО КАБИНЕТА СТУДЕНТА И РАСПИСАНИЯ БГУИР**

БГУИР КП 1-40 04 01 033 ПЗ

Студент

Н. С. Сенько

Руководитель

А. В. Давыдчик

Минск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ОБЗОР СУЩЕСТВУЮЩИХ АНАЛОГОВ	4
1.1 Облачные AI-генераторы тестов (Questgen.ai, Quizizz AI, Conker.ai)....	4
1.2 Традиционные системы управления обучением (Moodle, iSpring Suite)	5
2 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ И ИНСТРУМЕНТЫ РАЗРАБОТКИ	6
2.1 Функциональные требования к системе	6
2.2 Выбор СУБД	7
3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ	14
3.1 Инфологическая модель	14
3.2 Даталогическая модель	15
4 РАЗРАБОТКА БАЗЫ ДАННЫХ	18
4.1 Физическая модель базы данных	18
4.2 Создание таблиц	19
4.3 Запросы	20
5 ТЕСТИРОВАНИЕ РАБОТОСПОСОБНОСТИ БАЗЫ ДАННЫХ	21

ВВЕДЕНИЕ

В условиях стремительной цифровизации корпоративного обучения и образовательных процессов особую актуальность приобретает задача автоматизации трудоемких этапов, таких как создание и проверка оценочных материалов. Существующие подходы, основанные на ручном формировании тестов, требуют значительных временных и интеллектуальных ресурсов, что замедляет внедрение новых учебных программ и обновление существующих баз знаний.

Целью данного курсового проекта является проектирование и разработка реляционной базы данных для веб-сервиса, предназначенного для автоматизированной генерации тестовых материалов на основе текстовых данных с использованием больших языковых моделей (LLM). Проектируемая система призвана решить ключевые проблемы современных инструментов: обеспечить высокий уровень конфиденциальности за счет локального развертывания и повысить качество генерируемого контента благодаря адаптации AI-моделей под специфику предметной области.

Основные задачи разработки включают создание надежной и масштабируемой структуры хранения данных, способной обеспечить полный цикл работы с контентом: от загрузки исходных документов и хранения сгенерированных вопросов до управления тестовыми сессиями, назначениями и анализа результатов. Внедрение такой системы позволит существенно сократить трудозатраты методистов и преподавателей, ускорить процессы аттестации и онбординга, а также повысить объективность оценки знаний. Проект направлен на создание фундаментальной основы для современного и безопасного инструмента, который решает ключевые проблемы ручного создания тестов в корпоративной и академической среде.

В рамках данной работы будут выполнены следующие задачи:

- 1 Анализ существующих аналогов и выявление их достоинств и недостатков.
- 2 Формирование функциональных требований и выбор подходящих инструментов для разработки.
- 3 Проектирование базы данных, включая разработку инфологической и даталогической моделей.
- 4 Создание физической базы данных с использованием выбранной СУБД.
- 5 Проведение тестирования для оценки работоспособности и целостности базы данных.

Центральным элементом данной работы является проектирование базы данных, которая должна гарантировать целостность информации, поддерживать сложные взаимосвязи между пользователями, контентом и результатами тестирования, а также обеспечивать высокую производительность при выполнении запросов.

1 ОБЗОР СУЩЕСТВУЮЩИХ АНАЛОГОВ

Перед началом проектирования автоматизированной системы необходимо провести анализ существующих на рынке решений, которые реализуют схожий функционал. Этот процесс позволяет выявить их сильные и слабые стороны, оценить степень удовлетворения потребностей корпоративного сектора, а также определить ключевые направления для улучшения и внедрения уникальных возможностей. Проведенный обзор способствует выбору оптимальных архитектурных и технологических решений при разработке собственной системы и помогает избежать недостатков, присущих существующим продуктам. На рынке можно выделить две основные категории аналогов: облачные сервисы для AI-генерации тестов и традиционные системы управления обучением (LMS).

1.1 Облачные AI-генераторы тестов (Questgen.ai, Quizizz AI, Conker.ai)

Данная категория представляет собой SaaS-платформы, которые используют искусственный интеллект для автоматического создания тестов. Принцип их работы заключается в том, что пользователь загружает исходный материал в виде текста, документа или ссылки на веб-ресурс, после чего облачный сервис на базе универсальных больших языковых моделей, таких как GPT, генерирует набор вопросов и вариантов ответов.

Преимущества таких систем заключаются в их доступности и простоте использования. Они не требуют установки и настройки, имеют интуитивно понятный интерфейс и позволяют получить результат в кратчайшие сроки. Это делает их удобным инструментом для решения несложных образовательных задач, не требующих обработки конфиденциальной информации.

Однако для корпоративного использования такие сервисы имеют ряд критических недостатков. Во-первых, главным барьером является проблема безопасности: все исходные материалы, которые могут содержать коммерческую тайну, персональные данные или другую чувствительную информацию, передаются и обрабатываются на сторонних серверах, что создает прямой и неприемлемый риск утечки данных. Во-вторых, качество генерируемого контента часто бывает поверхностным. Универсальные языковые модели не способны понять узкоспециализированный контекст, внутреннюю терминологию и нюансы корпоративных материалов, что приводит к созданию некорректных или методологически слабых вопросов. Наконец, работа по подписной модели приводит к постоянным операционным

расходам, которые могут стать значительными при большом количестве пользователей.

Таким образом, облачные генераторы являются удобным, но ограниченным инструментом, не подходящим для серьезных корпоративных задач, где безопасность и релевантность контента стоят на первом месте.

1.2 Традиционные системы управления обучением (Moodle, iSpring Suite)

Вторую категорию аналогов составляют классические системы управления обучением, или LMS. Это мощные и комплексные платформы, предназначенные для организации, проведения и контроля всего учебного процесса в компании или учебном заведении. Они предоставляют широкий функционал для размещения курсов, назначения обучения, отслеживания прогресса и коммуникации.

Основным преимуществом данных систем является их комплексный подход к управлению обучением. Они позволяют выстроить полноценную образовательную экосистему и являются отраслевым стандартом во многих организациях. Некоторые современные LMS начинают интегрировать элементы искусственного интеллекта, однако, как правило, это сводится к интеграции с теми же облачными сервисами, что и в первой категории, с сохранением всех их недостатков.

Ключевым недостатком традиционных LMS в контексте нашего проекта является то, что они не решают основную проблему — автоматизацию создания контента. Конструкторы тестов в подавляющем большинстве таких систем являются полностью ручными. Они предоставляют платформу для размещения и проведения тестирования, но вся работа по изучению материалов, составлению вопросов и вариантов ответов ложится на плечи методистов и экспертов. Это делает процесс создания тестов таким же трудоемким и длительным, как и без использования LMS.

Проведенный анализ показал, что на рынке существует незанятая ниша для продукта, который бы объединил в себе автоматизацию создания контента с помощью AI и строгие требования корпоративного сектора к безопасности данных. Ни одно из существующих решений не предлагает локального развертывания интеллектуального генератора тестов, адаптированного под специфику клиента. Это подтверждает актуальность и востребованность разработки веб-сервиса, который предоставит компаниям полный контроль над своими данными и высокое качество оценочных материалов.

2 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ И ИНСТРУМЕНТЫ РАЗРАБОТКИ

2.1 Функциональные требования к системе

Основной задачей разрабатываемого веб-сервиса является комплексная автоматизация процесса тестирования в корпоративной среде, начиная от создания оценочных материалов и заканчивая анализом результатов. Система должна предоставлять безопасный, интуитивно понятный и эффективный инструмент для двух ключевых категорий пользователей: администраторов, отвечающих за контент, и сотрудников, проходящих тестирование.

Ключевые функциональные требования к системе определяются на основе ролевой модели доступа. Для роли «Администратор», которую выполняют специалисты по обучению, методисты или руководители, должен быть реализован полный набор инструментов для управления контентом и тестированием. Это включает возможность безопасной загрузки в систему исходных текстовых материалов в форматах .txt и .md, на основе которых будет происходить генерация вопросов. Администратор должен иметь возможность запускать асинхронную задачу по интеллектуальной обработке документов, а после ее завершения — просматривать, редактировать или удалять сгенерированный пул вопросов и ответов. Также должна быть предусмотрена функция ручного создания вопросов для максимальной гибкости. На основе сформированного пула вопросов администратор должен уметь создавать тестовые сессии, устанавливать для них параметры, такие как ограничение по времени или количество попыток, и назначать их конкретным сотрудникам или целым отделам.

Для роли «Пользователь», то есть рядового сотрудника, система должна предоставлять простой и удобный интерфейс для прохождения аттестации. Пользователь должен иметь доступ к личному кабинету, где отображается список всех назначенных ему тестов. Процесс прохождения тестирования должен быть реализован в интерактивном формате, а результат — автоматически рассчитываться и отображаться сразу после завершения попытки. Кроме того, система должна хранить историю всех пройденных тестов, предоставляя пользователю доступ к архиву своих результатов для самоанализа.

Для обеспечения аналитических функций система должна предоставлять администратору доступ к сводной статистике по успеваемости. Необходимо реализовать дашборды и отчеты, которые позволят оценивать уровень знаний как отдельных сотрудников, так и целых отделов, анализировать сложность вопросов и отслеживать общий прогресс в обучении. Таким образом,

реализация перечисленных функциональных требований позволит создать целостную и самодостаточную систему, закрывающую все потребности корпоративного обучения в области создания и проведения тестирования.

2.2 Выбор СУБД

Для реализации разрабатываемого программного продукта выбран подход, основанный на использовании реляционной модели данных. Это решение обусловлено необходимостью структурированного хранения и обработки большого количества взаимосвязанных данных, включая сведения о пользователях, расписаниях, учебных дисциплинах, результатах успеваемости и заявках студентов. Реляционная структура обеспечивает логическую целостность данных, удобство их представления в виде таблиц и эффективность при выполнении запросов различной сложности.

Выбор архитектуры и типа СУБД определяется задачами системы и особенностями её функционирования как в онлайн-, так и в офлайн-режиме. Хранилище данных должно поддерживать возможность локального хранения информации на клиентском устройстве и синхронизации с серверной базой при наличии интернет-соединения. Такой подход позволяет обеспечить стабильную работу системы, доступность информации и целостность данных даже при временном отсутствии сети.

Требования к СУБД:

1 Поддержка работы с большими объемами данных. Система должна корректно функционировать при хранении значительных объёмов информации о пользователях, расписаниях и учебных процессах. Важно обеспечить высокую скорость выполнения операций чтения, записи и обновления данных без существенной потери производительности.

2 Гарантия целостности и согласованности данных. СУБД должна поддерживать транзакционность и выполнять ACID-свойства, что гарантирует корректность операций при одновременном обращении множества пользователей и в случае возможных сбоев соединения или ошибок приложения.

3 Высокая производительность при обработке запросов. Приложение активно использует выборки и фильтрацию по множеству параметров – дате, преподавателю, дисциплине и т.д. СУБД должна обеспечивать эффективное выполнение таких запросов и минимизировать время отклика при поиске данных.

4 Масштабируемость и расширяемость. Архитектура базы данных должна позволять увеличение объёма хранимых данных и количества пользователей без потери производительности. При необходимости должна

быть возможность распределённого хранения данных или расширения функционала без переработки структуры.

5 Поддержка синхронизации данных. Система должна обеспечивать двусторонний обмен информацией между клиентской и серверной частью. При восстановлении интернет-соединения СУБД должна корректно обрабатывать обновления, разрешая возможные конфликты данных.

6 Поддержка офлайн-доступа. Необходимо предусмотреть возможность локального хранения данных на устройстве пользователя. Это позволит студентам работать с ранее загруженной информацией при отсутствии интернет-соединения.

7 Обеспечение безопасности данных. СУБД должна предоставлять средства защиты персональной информации, включая контроль доступа, шифрование и безопасное хранение учетных данных. Должна быть реализована защита от несанкционированного доступа и утечки данных.

8 Минимальные требования к вычислительным ресурсам. Так как приложение ориентировано на мобильные устройства, важно, чтобы выбранная система управления базами данных имела низкие требования к памяти и процессорным ресурсам, не влияя на стабильность работы приложения.

9 Надёжность и отказоустойчивость. Система хранения данных должна обеспечивать стабильную работу даже при сбоях и внезапных отключениях, а также гарантировать сохранение и восстановление информации после сбоев.

Таким образом, выбранный подход к организации хранения данных обеспечивает надёжность, безопасность и эффективность функционирования системы. Он позволяет реализовать все ключевые функции приложения, обеспечивая стабильный доступ к информации, поддержку офлайн-режима и возможность дальнейшего масштабирования без снижения производительности.

Рассмотрим самый популярный SQL базы данных.

2.2.1 PostgreSQL – одна из наиболее мощных и гибких реляционных СУБД с открытым исходным кодом. Её разработка началась в 1986 году профессором Майклом Стоунбрейкером (Университет Калифорнии в Беркли) под названием POSTGRES. Система создавалась для изучения теоретических концепций реляционных баз данных. В 1996 году проект переименовали в PostgreSQL, и с тех пор он развивается как открытый инструмент для хранения и обработки больших массивов данных.

Преимущества:

1 Высокая производительность. PostgreSQL эффективно работает с большими объёмами данных и сложными запросами благодаря индексации, оптимизации запросов и продуманной обработке крупных таблиц. Для

агрегатора автобусных маршрутов и билетов это означает стабильную работу при пиковых нагрузках – например, при массовом обработке транзакций и запросов.

2 Поддержка транзакций и ACID-свойств. Система гарантирует целостность данных, что критично для бронирования билетов: исключаются ошибки при параллельных попытках занять одно место. Транзакции обеспечивают надёжность и безопасность данных.

3 Работа со сложными типами данных. СУБД поддерживает JSON, массивы, хранимые процедуры и пользовательские типы. Это удобно для хранения неструктурированных данных – например, информации о маршрутах и местах в автобусах. В агрегаторе такая функциональность полезна для динамического управления расписаниями.

4 Открытая лицензия. PostgreSQL распространяется под лицензией Open Source – без лицензионных платежей. Это снижает затраты, особенно для стартапов и компаний с ограниченным бюджетом, и даёт гибкость в разработке.

Недостатки:

1 Требования к квалификации. Для оптимальной настройки (индексы, параллельные запросы, репликация) нужны глубокие знания. Без опыта возможны проблемы с производительностью и надёжностью.

2 Сложность развёртывания. По сравнению с MySQL или SQLite, PostgreSQL требует больше времени и усилий на настройку масштабируемости, репликаций, резервного копирования.

2.2.2 MySQL – популярная реляционная СУБД с открытым исходным кодом. Разработку начала в 1994 году шведская компания MySQL AB (основатели – Михаэль Видениус и Альф Ларссон). Система создавалась для работы с большими данными в интернет-приложениях. В 2008 году компанию приобрела Sun Microsystems, а в 2010 году – Oracle Corporation. При этом MySQL остаётся одной из самых востребованных СУБД.

Преимущества:

1 Простота использования. MySQL легко установить и настроить, что важно для быстрого запуска проектов. В отличие от PostgreSQL, здесь не требуется углублённая конфигурация – это делает СУБД подходящей для стартапов и небольших систем.

2 Репликация и масштабируемость. Система предлагает надёжную репликацию для создания резервных копий, распределения нагрузки между серверами, обеспечения высокой доступности. Для агрегатора это значит стабильную работу при росте числа пользователей и объёма данных.

3 Активное сообщество. У MySQL большое сообщество разработчиков, множество готовых библиотек и документации. Это упрощает решение

проблем и снижает затраты на поддержку, делая систему доступной для специалистов разного уровня.

Недостатки:

1 MySQL демонстрирует определённые ограничения при работе с масштабными массивами данных – в этом аспекте система уступает PostgreSQL. Несмотря на в целом достойную производительность, MySQL менее эффективен при обработке объёмных наборов информации и выполнении сложных запросов, требующих многоэтапных выборок или интенсивной транзакционной нагрузки. Для проекта агрегатора автобусных маршрутов это может проявиться в замедлении ключевых операций: например, при поиске рейсов по совокупности критериев (дата, направление, ценовой диапазон) или при расчёте стоимости билетов с учётом динамически изменяющихся тарифов. В ситуациях пиковой нагрузки – например, при массовом оформлении бронирований – система может не обеспечить ту же скорость отклика, которую даёт PostgreSQL в аналогичных условиях.

2 Ограниченная поддержка сложных типов данных и некоторых расширений. MySQL имеет ограниченную поддержку для работы с более сложными типами данных, такими как JSON или массивы, по сравнению с PostgreSQL.

Таким образом, MySQL представляет собой отличное решение для простых и средних по сложности проектов, где важна высокая доступность, масштабируемость и быстрая настройка.

2.2.3 MariaDB – это реляционная система управления базами данных с открытым исходным кодом, которая является форком (ответвлением) MySQL. Проект был основан в 2009 году Майклом «Монти» Видениусом, одним из оригинальных разработчиков MySQL, после того как Oracle Corporation приобрела Sun Microsystems, владевшую правами на MySQL. Основной мотивацией создания MariaDB стало желание сохранить базу данных полностью свободной и открытой под лицензией GNU GPL, гарантируя ее развитие силами независимого сообщества разработчиков.

Преимущества:

1 Полная совместимость с MySQL. MariaDB изначально проектировалась как «drop-in replacement» (прямая замена) для MySQL. Это означает, что миграция с MySQL на MariaDB проходит без необходимости изменять код приложения. Для разрабатываемого веб-сервиса это критически важно, так как позволяет использовать стандартные драйверы и инструменты без дополнительной настройки.

2 Повышенная производительность и новые возможности. MariaDB включает в себя ряд улучшений по сравнению с MySQL, в том числе более совершенный оптимизатор запросов и дополнительные движки хранения, такие как Aria, который является более отказоустойчивой альтернативой

MyISAM. В контексте системы тестирования это обеспечивает более быструю обработку результатов, формирование аналитических отчетов для администраторов и высокую скорость отклика при одновременном прохождении тестов множеством пользователей.

3 Гарантированная открытость и активное сообщество. В отличие от MySQL, чья коммерческая версия контролируется Oracle, MariaDB развивается полностью открытым сообществом под эгидой MariaDB Foundation. Это гарантирует, что СУБД останется бесплатной и свободной от корпоративных ограничений. Для проекта это означает отсутствие лицензионных отчислений, прозрачный процесс разработки и оперативное получение обновлений безопасности.

Недостатки:

1 Меньшая корпоративная поддержка по сравнению с Oracle MySQL. Несмотря на наличие коммерческой поддержки от MariaDB Corporation и других компаний, экосистема корпоративной поддержки Oracle для MySQL является более обширной и устоявшейся. Для крупных предприятий, требующих сертифицированных решений и официальных контрактов на поддержку, это может стать фактором выбора в пользу продукта Oracle.

2 Риск расхождения с MySQL в будущем. Хотя на данный момент совместимость сохраняется на высоком уровне, с течением времени MariaDB и MySQL развиваются независимо друг от друга. Существует долгосрочный риск, что различия в функционале станут настолько значительными, что простая миграция между системами будет невозможна. Это может усложнить долгосрочное планирование и поддержку проекта, если возникнет необходимость вернуться на MySQL.

2.2.4 Oracle Database – это одна из самых мощных и широко используемых коммерческих реляционных систем управления базами данных, разработанная компанией Oracle Corporation. История Oracle началась в 1977 году, когда Ларри Эллисон, Боб Минтер и Эд Оутс основали компанию Relational Software Inc., которая позже была переименована в Oracle Corporation. В 1979 году Oracle представила первую коммерческую реляционную СУБД, которая использовала язык запросов SQL (Structured Query Language). С тех пор Oracle стал лидером на рынке корпоративных СУБД, с особым акцентом на высокую производительность, надежность и масштабируемость, что сделало Oracle

Database стандартом для крупных организаций и финансовых учреждений.

Преимущества:

1 Высокая производительность и отличная масштабируемость. Oracle Database предоставляет высокую производительность, особенно для крупных

корпоративных приложений, работающих с большими объемами данных. Система способна обрабатывать огромные нагрузки и работать с большими базами данных, что делает ее идеальной для систем, которые требуют высокой скорости обработки данных при выполнении сложных транзакций и запросов. Это преимущество особенно важно для крупных агрегаторов с большим количеством пользователей и сложной структурой данных.

2 Oracle Database известна своей надежной поддержкой транзакций и соблюдением ACID-свойств, что гарантирует целостность данных. Это особенно важно в контексте системы бронирования, где важно обеспечить, чтобы данные о бронированиях и оплатах были точными и не были потеряны при сбоях системы. Также поддержка репликации данных позволяет создавать высокодоступные решения, обеспечивая бесперебойную работу системы.

3 Широкий спектр функциональных возможностей для бизнеса. Oracle Database предоставляет множество инструментов для бизнес-анализа, работы с большими данными, машинного обучения и аналитики. Это делает систему универсальным инструментом для предприятий, которые могут использовать эти функции для углубленного анализа данных и построения бизнесотчетности. Для проекта агрегатора автобусных маршрутов это может быть полезно, если система будет расширяться с внедрением аналитики для прогнозирования спроса на билеты или изучения поведения пользователей.

Недостатки:

1 Высокая стоимость лицензирования. Одним из главных недостатков Oracle Database является высокая стоимость лицензирования, что делает ее дорогим решением для малых и средних бизнесов. Стоимость лицензий и ежегодной поддержки может быть значительным бременем для стартапов и небольших компаний, особенно если они не используют все возможности СУБД. В контексте проекта агрегатора автобусных маршрутов и билетов это может означать большие затраты на эксплуатацию системы, которые могут быть не оправданы для небольшого бизнеса.

2 Требуется значительных ресурсов для развертывания и эксплуатации. Развертывание и эксплуатация Oracle Database требуют значительных вычислительных и человеческих ресурсов. Это включает в себя не только покупку и установку серверов, но и поддержку на протяжении всего срока эксплуатации. Сложная настройка и оптимизация базы данных могут потребовать высококвалифицированных специалистов, что увеличивает затраты на техническую поддержку. Для небольших проектов это может быть чрезмерным, особенно если задачи можно решить с помощью более легких и дешевых СУБД.

3 Не является бесплатной. В отличие от PostgreSQL или MySQL, Oracle Database является платной СУБД с ограниченной бесплатной версией. Стоимость лицензирования может быть высока, что увеличивает затраты на

проект и делает решение менее привлекательным для стартапов и малых компаний. Для агрегации автобусных маршрутов и билетов, где высокая стоимость лицензии может не быть оправдана функциональностью, доступной в других СУБД, использование Oracle Database может быть нецелесообразным.

Таким образом, MariaDB представляет собой превосходное решение для проектов, где важны производительность, открытость и независимость от корпораций, при этом сохраняя полную совместимость с одной из самых популярных СУБД в мире.

3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

Проектирование базы данных является важным этапом в разработке системы агрегатора автобусных маршрутов и билетов, так как правильная структура данных определяет эффективность работы системы, обеспечивая быструю обработку запросов, целостность и безопасность данных. В рамках этого раздела будут рассмотрены этапы проектирования, включая инфологическую и даталогическую модели, модель миграции данных, а также требования к входным данным и форматы их хранения.

3.1 Инфологическая модель

Инфологическая модель – это этап проектирования базы данных, на котором происходит детальное описание структуры данных, их взаимосвязей и организационных единиц системы. В рамках этого этапа создается концептуальная модель, отражающая основные объекты и связи между ними. Инфологическая модель не зависит от конкретной реализации в СУБД, она служит основой для дальнейшего проектирования и реализации базы данных, позволяя понять, как данные будут структурированы и взаимодействовать в системе.

В данной работе инфологическая модель базы данных отражает структуру и связи между сущностями, необходимыми для разработки агрегатора автобусных маршрутов и билетов. Основными сущностями являются пользователи, маршруты, билеты, транзакции лояльности и различные связанные с ними данные. Важно, чтобы модель была гибкой и могла эффективно обрабатывать запросы, касающиеся маршрутов, бронирований, лояльности и поддержки пользователей.

Инфологическая модель базы данных включает в себя следующие сущности и их атрибуты:

1 Пользователь (User) – ключевая сущность, представляющая любого зарегистрированного участника системы. Атрибуты включают идентификатор, полное имя, электронную почту и хэш пароля.

2 Роль (Role) – справочная сущность, определяющая уровень доступа пользователя к функциям системы. Основные роли: «Администратор» и «Пользователь». Один пользователь может иметь одну или несколько ролей.

3 Группа (Group) – сущность для объединения пользователей, например, по отделам или учебным потокам. Это позволяет упростить назначение тестов. Один пользователь может состоять в нескольких группах, и в одной группе может быть много пользователей.

4 Исходный документ (Source Document) – представляет текстовый материал, загруженный администратором для последующей генерации

вопросов. Атрибуты включают название файла, путь к нему на сервере, статус обработки и ссылку на пользователя, который его загрузил.

5 Вопрос (Question) – центральная сущность, хранящая текст вопроса. Каждый вопрос может быть сгенерирован на основе одного исходного документа или создан вручную.

6 Вариант ответа (Answer Option) — представляет один из вариантов ответа на конкретный вопрос. Каждый вопрос имеет несколько вариантов ответов, один или несколько из которых помечены как правильные.

7 Тест (Test) — логическая сущность, объединяющая набор вопросов в единый оценочный материал. Тест имеет такие атрибуты, как название, описание, ограничение по времени и количество попыток. Один тест может включать множество вопросов, и один и тот же вопрос может входить в разные тесты.

8 Назначение теста (Test Assignment) — фиксирует факт назначения определенного теста конкретному пользователю или целой группе.

9 Сессия тестирования (Test Session) — представляет собой одну конкретную попытку прохождения теста пользователем. Хранит информацию о статусе (начата, завершена), времени начала и окончания, а также итоговом балле.

10 Ответ пользователя (User Answer) — фиксирует, какой вариант ответа был выбран пользователем на конкретный вопрос в рамках определенной сессии тестирования.

Эти сущности связаны между собой логическими отношениями: «один-ко-многим» (например, один пользователь может пройти много сессий) и «многие-ко-многим» (например, один тест может содержать много вопросов, и один вопрос может входить во много тестов). Инфологическая модель служит прочным фундаментом для перехода к следующему, более детализированному этапу проектирования.

3.2 Даталогическая модель

Даталогическая модель описывает структуру данных на уровне базы данных, в том числе типы данных, ограничения и связи между таблицами. Она отвечает за определение точных типов данных для каждого атрибута сущности, установление ограничений на значения и обеспечение целостности данных. В данном разделе представлена детальная спецификация всех сущностей, их атрибутов и типов данных, которые будут использоваться в базе данных системы. Модель строится на основе инфологической модели, уточняя типы данных и устанавливая точные форматы хранения информации для эффективного функционирования системы.

Описание таблиц и их атрибутов:

1 roles (Роли)

- id (INT, PRIMARY KEY) – Уникальный идентификатор роли.
- name (VARCHAR) – Название роли (например, "admin", "user").
- users (Пользователи)
- id (BIGINT, PRIMARY KEY) – Уникальный идентификатор пользователя.
- full_name (VARCHAR) – Полное имя.
- email (VARCHAR, UNIQUE) – Электронная почта.
- password_hash (VARCHAR) – Хэш пароля.
- created_at (TIMESTAMP) -Время создания записи.

2 groups (Группы)

- id (BIGINT, PRIMARY KEY) - Уникальный идентификатор группы.
- name (VARCHAR) - Название группы/отдела.
- source_documents (Исходные документы)
- id (BIGINT, PRIMARY KEY) - Идентификатор документа.
- filename (VARCHAR) - Имя файла.
- status (ENUM) - Статус обработки ('pending', 'processing', 'completed', 'failed').
- uploader_id (BIGINT, FOREIGN KEY to users.id) - ID пользователя-загрузчика.
- created_at (TIMESTAMP) - Время загрузки.

3 questions (Вопросы)

- id (BIGINT, PRIMARY KEY) - Идентификатор вопроса.
- question_text (TEXT) - Текст вопроса.
- source_document_id (BIGINT, FOREIGN KEY to source_documents.id) - Ссылка на исходный документ.
- creator_id (BIGINT, FOREIGN KEY to users.id) - ID создателя.

4 answer_options (Варианты ответов)

- id (BIGINT, PRIMARY KEY) - Идентификатор варианта.
- question_id (BIGINT, FOREIGN KEY to questions.id) - Ссылка на вопрос.
- answer_text (TEXT) - Текст ответа.
- is_correct (BOOLEAN) - Признак правильного ответа.

5 tests (Тесты)

- id (BIGINT, PRIMARY KEY) - Идентификатор теста.
- title (VARCHAR) - Название теста.
- creator_id (BIGINT, FOREIGN KEY to users.id) - ID создателя.
- time_limit_minutes (INT) - Ограничение по времени.

6 test_assignments (Назначения тестов)

- id (BIGINT, PRIMARY KEY) - Идентификатор назначения.
- test_id (BIGINT, FOREIGN KEY to tests.id) - Ссылка на тест.
- user_id (BIGINT, FOREIGN KEY to users.id) - Ссылка на пользователя (для персонального назначения).
- group_id (BIGINT, FOREIGN KEY to groups.id) - Ссылка на группу (для группового назначения).

7 test_sessions (Сессии тестирования)

- id (BIGINT, PRIMARY KEY) - Идентификатор сессии.
- test_id (BIGINT, FOREIGN KEY to tests.id) - Ссылка на тест.
- user_id (BIGINT, FOREIGN KEY to users.id) - Ссылка на пользователя.
- status (ENUM) - Статус ('in_progress', 'completed').
- started_at (TIMESTAMP) - Время начала.
- completed_at (TIMESTAMP) - Время завершения.
- score (DECIMAL) - Итоговый балл.

8 user_answers (Ответы пользователя)

- id (BIGINT, PRIMARY KEY) - Идентификатор ответа.
- test_session_id (BIGINT, FOREIGN KEY to test_sessions.id) - Ссылка на сессию.
- question_id (BIGINT, FOREIGN KEY to questions.id) - Ссылка на вопрос.
- selected_option_id (BIGINT, FOREIGN KEY to answer_options.id) - Ссылка на выбранный вариант.

Создание даталогической модели представляет собой ключевой этап в проектировании базы данных, поскольку именно она формирует фундамент для последующей реализации системы. В рамках данного раздела была разработана подробная структура данных: для каждой сущности чётко прописаны атрибуты с указанием конкретных типов данных и наложенных ограничений, а также заданы взаимосвязи между таблицами. Подобный подход способствует поддержанию целостности и согласованности информации, минимизируя вероятность ошибок при работе с базой данных. Полученная модель служит базовой основой для проектирования физической базы данных и дальнейшего масштабирования системы. Грамотное определение типов данных и характера взаимосвязей между сущностями существенно оптимизирует производительность операций с данными и повышает общую надёжность системы.

4 РАЗРАБОТКА БАЗЫ ДАННЫХ

Разработка базы данных является ключевым этапом создания программной системы, обеспечивая основу для хранения, обработки и анализа информации. На данном этапе осуществляется переход от логической модели данных, отражающей концептуальную структуру проекта, к физической модели, реализующей её средствами реляционной базы данных, в нашем случае – MariaDB.

База данных предназначена для централизованного хранения сведений о пользователях, профилях, уведомлениях, расписаниях занятий, учебных группах, справках, успеваемости и других элементах системы. Она обеспечивает взаимосвязанность всех сущностей, целостность данных, а также возможность выполнения аналитических запросов и автоматизированных процедур.

База данных предназначена для централизованного хранения всей информации, необходимой для функционирования веб-сервиса: от учетных записей пользователей и загруженных документов до сгенерированных вопросов, ответов, тестовых сессий и их результатов. Правильная реализация физического уровня является залогом стабильности, производительности и безопасности всего приложения.

4.1 Физическая модель базы данных

Физическая модель базы данных представляет собой конечное описание структуры хранения данных. Она детализирует даталогическую модель, уточняя типы данных в соответствии со спецификацией MariaDB, определяя параметры полей, такие как UNSIGNED для числовых идентификаторов, DEFAULT значения и правила поведения при обновлении или удалении связанных записей (ON DELETE CASCADE, ON DELETE SET NULL).

В рамках проекта были разработаны таблицы для всех ключевых сущностей: users, roles, groups, source_documents, questions, answer_options, tests, test_assignments, test_sessions и user_answers. Для реализации связей "многие-ко-многим" были созданы промежуточные таблицы user_roles, user_groups и test_questions.

Каждая основная таблица имеет первичный ключ (PRIMARY KEY), как правило, на поле id с атрибутом AUTO_INCREMENT, что гарантирует уникальность каждой записи и автоматизирует генерацию идентификаторов. Внешние ключи (FOREIGN KEY) используются для строгого контроля ссылочной целостности, предотвращая появление "осиротевших" записей. Например, невозможно удалить пользователя, если на него ссылаются записи

о загруженных им документах или созданных тестах (если не задано каскадное удаление). Для ускорения операций поиска и соединения таблиц были добавлены индексы на поля, которые часто используются в условиях WHERE и JOIN, например, на email в таблице users или на status в таблицах source_documents и test_sessions.

4.2 Создание таблиц

Создание таблиц было выполнено с помощью скриптов на языке SQL (DDL - Data Definition Language), которые полностью описывают физическую структуру базы данных. Для каждого поля были выбраны оптимальные типы данных для минимизации занимаемого дискового пространства и повышения производительности.

Пример создания таблицы вопросов:

```
CREATE TABLE questions (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `question_text` TEXT NOT NULL COMMENT 'Текст вопроса',  
  `source_document_id` BIGINT UNSIGNED NULL COMMENT 'Ссылка на документ, из которого сгенерирован вопрос (NULL для ручного)',  
  `creator_id` BIGINT UNSIGNED NOT NULL COMMENT 'ID пользователя, который инициировал генерацию или создал вопрос',  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `fk_question_document` FOREIGN KEY  
  (`source_document_id`) REFERENCES `source_documents` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `fk_question_creator` FOREIGN KEY (`creator_id`) REFERENCES `users` (`id`) ON DELETE CASCADE  
)  
COMMENT='Пул вопросов для тестов';
```

В данном примере видно, что для идентификаторов используется тип `BIGINT UNSIGNED` для поддержки большого количества записей. Поля `created\_at` и `updated\_at` автоматически отслеживают время создания и последнего изменения записи. Связи с таблицами `source\_documents` и `users` установлены через внешние ключи с определением соответствующего поведения: при удалении документа ссылка на него в вопросе станет `NULL`, а при удалении пользователя будут удалены и все созданные им вопросы.

Для всех таблиц реализованы связи по ключевым полям, что обеспечивает структурную целостность данных и предотвращает появление «осиротевших» записей.

4.3 Запросы

Запросы к базе данных (DML - Data Manipulation Language) являются основным инструментом взаимодействия приложения с данными. Они обеспечивают реализацию всей бизнес-логики: от аутентификации пользователя до генерации сложных аналитических отчетов.

Примеры ключевых запросов, используемых в системе:

1 Получение списка тестов, назначенных конкретному пользователю, включая те, что назначены его группам:

```
SELECT t.id, t.title, t.description
FROM tests t
JOIN test_assignments ta ON t.id = ta.test_id
WHERE ta.user_id = 123 OR ta.group_id IN (SELECT group_id FROM
user_groups WHERE user_id = 123);
```

2 Выборка всех вопросов с вариантами ответов для определенного теста:

```
SELECT q.id, q.question_text, ao.id, ao.answer_text, ao.is_correct
FROM questions q
JOIN answer_options ao ON q.id = ao.question_id
JOIN test_questions tq ON q.id = tq.question_id
WHERE tq.test_id = 1;
```

3 Сохранение ответа пользователя в рамках сессии тестирования:

```
INSERT INTO user_answers (test_session_id, question_id,
selected_option_id)
VALUES (987, 55, 210);
```

4 Подсчет итогового балла для завершенной сессии тестирования:

```
SELECT (SUM(CASE WHEN ao.is_correct = 1 THEN 1 ELSE 0 END) /
COUNT(ua.id)) * 100 AS final_score
FROM user_answers ua
JOIN answer_options ao ON ua.selected_option_id = ao.id
WHERE ua.test_session_id = 987;
```

В результате разработки физической базы данных была создана надежная, нормализованная и эффективная структура, полностью готовая к интеграции с серверной частью приложения. Она обеспечивает целостность данных, высокую производительность и логическую непротиворечивость, что является необходимым условием для стабильной работы веб-сервиса. Использование реляционной модели позволило эффективно описать все взаимосвязи между элементами системы и обеспечить их согласованность при выполнении любых операций.

5 ТЕСТИРОВАНИЕ РАБОТОСПОСОБНОСТИ БАЗЫ ДАННЫХ